



TOPIC : JVM ARCHITECTURE

JVM Architecture

Java प्रोग्राम चालवण्यासाठी **JVM** हा सर्वात महत्वाचा भाग आहे.

JVM म्हणजे **Java Virtual Machine** जी **Java bytecode execute** करते.

JVM हे एक **virtual machine** आहे जे **"Write Once, Run Anywhere"** ही **Java** ची खासियत **possible** करते.

JVM Working Flow

1. **Java Source Code (.java file)**
→ **Compiler** → **Bytecode (.class file)**
2. **Bytecode JVM** ला दिलं जातं
→ **JVM** त्याला **machine-understandable instructions** मध्ये **convert** करून **execute** करतं.

JVM Architecture Components

JVM मुख्यतः 3 मोठ्या **parts** मध्ये **divide** केलेलं आहे:

1. **Class Loader Subsystem (क्लास लोडर सबसिस्टम)**
 - **Responsibility:** **.class files load** करणं.
 - **JVM** ला **bytecode** समजण्यासाठी सर्वप्रथम हा **part** काम करतो.
 - याचे 3 **main phases** आहेत:
 1. **Loading** - **.class files memory** मध्ये **load** करणे.
 2. **Linking - Verification** (**bytecode correct** आहे का?), **Preparation** (**memory allocate** करणे), **Resolution** (संदर्भ **resolve** करणे).
 3. **Initialization** - **Static variables initialize** करणे आणि **static blocks execute** करणे.

2. Runtime Data Areas (रनटाईम डेटा एरिया)

हा **JVM** चा **main memory** भाग आहे, जिथे **execution** दरम्यान सगळी माहिती **store** होते.

मुख्य **divisions:**

(a) Method Area

- **Class** ची **metadata**, **static variables**, **constant pool** इथे **store** होतं.
- **Example:** जर **class** मध्ये **static int count = 0** असेल, तर तो **Method Area** मध्ये असेल.

(b) Heap Area

- **Objects** आणि **instance variables** इथे **store** होतात.
- **JVM garbage collector unused objects** इथून हटवतो.

(c) Stack Area

- प्रत्येक **thread** चा स्वतःचा **stack** असतो.
- **Stack** मध्ये **frames** असतात → प्रत्येक **frame method call** साठी तयार होतं.
 - **Local variables**
 - **Operand stack**
 - **Return values**

(d) Program Counter (PC Register)

- प्रत्येक **thread** साठी स्वतंत्र **register** असतो.
- सध्या कोणती **instruction** चालू आहे हे इथे **store** होतं.

(e) Native Method Stack

- **Java** व्यतिरिक्त इतर **languages** (जसे **C/C++**) मधल्या **native methods** ला **execute** करण्यासाठी वापरलं जातं.

3. Execution Engine (एक्झिक्युशन इंजिन)

Bytecode ला **actual machine code** मध्ये **convert** करून **CPU execute** करू शकतो असा बनवणं.

Components:

1. **Interpreter** - **Bytecode** एकेक **instruction translate** करून चालवतो (**slower**).
2. **JIT Compiler (Just-In-Time)** - **Repeated code compile** करून **fast machine code** मध्ये **convert** करतो (**faster**).
3. **Garbage Collector** - **Heap** मधल्या **unused objects remove** करून **memory free** करतो.

4. Native Method Interface (JNI)

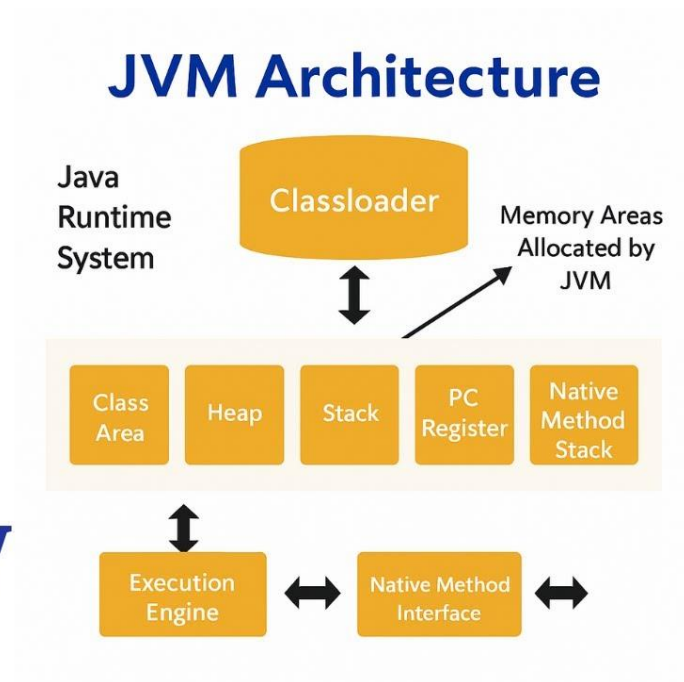
- **JVM** ला **OS-specific libraries (C, C++)** वापरता याव्यात म्हणून एक **bridge** साखं काम करतं.

5. Native Method Libraries

- **JNI** ने वापरलेल्या **actual libraries**.
-

JVM Architecture Diagram

How JVM Works Internally



TECHNORBIT
INFOSYSTEMS
Educating For Better Tomorrow